prompting-explained

Please, Thank You, and Other Words the AI Ignored

A teaching tool that shows which phrases in your prompt changed an AI's answer, and which didn't.

Vihaan Manchanda · Duke University · Explainable AI

TRY IT ON YOUR PHONE
Scan to open the app
github.com/Vihaan8/prompting-explained

01 Summary

Why. Staff using AI tools at work get no feedback on which parts of their prompts the model acted on, so they can't tell good habits from bad ones.

What we built. A tool that removes one phrase at a time, reruns the prompt, and shows which phrases changed the result.

What we found. On 240 test prompts across 8 everyday tasks with one model, the phrases it ranked highest mattered more than random ones (score drop 0.30 vs 0.18), and a rerun picked the same top phrase 78% of the time.

Goal. A training tool companies load with their own tasks, so employees learn which prompting habits change results on their real work.

02 The problem

You see the answer, not what produced it. A chat tool doesn't show which of your phrases the model acted on.

Small wording changes can matter. In one of our prompts, "dont guess" made the AI speculate *more,* in all 5 runs.

Some mistakes give no warning. Asked to "turn this ticket into a bug report" for a public tracker, the AI included the customer's name in 2 of 3 runs.

03 Who it's for, and what we asked

People learning to prompt, teams that train staff on AI tools, and teams that review how AI is used.

RESEARCH QUESTION

Can removing phrases one at a time and rerunning explain, **faithfully** and **repeatably**, which parts of an everyday prompt changed an AI's answer?

04 Methods

Four steps. Terms marked * are explained in the purple notes.

1 Split the prompt into phrases

An AI splits the prompt and tags each phrase (task, audience, format, politeness). Code keeps each phrase exactly as typed. These are the pieces the tool perturbs*.

*** Perturbation:** changing part of an input to see how the output responds. LIME does this with single words.

2 Remove one phrase and rerun

The tool deletes one phrase, reruns the shorter prompt, and compares the answers. This is occlusion*. Phrases that change nothing alone are then removed together once, to catch phrases that cover for each other.

*** Occlusion (leave-one-out):** hide one part of the input, rerun, and measure the change.

3 Score every answer two ways

Against the task. Code grades each answer against the task's checklist, from 0 (misses everything) to 1 (passes everything).

Against your instructions. A second AI rates whether the answer did what each phrase asked*, such as "use a candy example", which code can't check.

*** AI as a judge:** one AI grading another's output. It makes mistakes, so we measured them (07).

4 Filter out noise

Every version runs more than once. A change counts only if it is bigger than the noise floor*. Smaller changes stay grey, and the tool says "not clear" instead of guessing.

*** Noise floor:** two standard errors of the AI's run-to-run variation for that phrase.

05 Worked example

Task: get the AI to explain inflation to a 10-year-old cousin. One prompt:

> Hi there! Explain inflation to my 10-year-old cousin. Use exactly 3 bullet points. Mention a candy bar in one of them. Use proper technical terms.

- **Green · "Explain inflation":** without it, the AI explained how the body uses food for energy.
- **Blue · "Mention a candy bar":** the AI did it, but the score was the same either way.
- **Red · "Use proper technical terms":** the AI used terms like "Consumer Price Index". Without it, the score rose from 0.40 to 0.78.
- **Grey · "Hi there!":** a greeting gives the AI nothing to act on.

On a fresh rerun, 5 of 6 verdicts repeated. The "without it" sentences are written by an AI and labelled as descriptions; every number is computed by code.

06 What worked

- **The ranking found the phrases that mattered, where wording mattered.** Removing the 2 top-ranked phrases dropped the score by 0.30 on average, against 0.18 for 2 random phrases (79 prompts, p = 0.0002). On the kid task: 0.77 vs 0.45.
- **Results mostly repeated.** A full rerun picked the same top phrase in 62 of 80 prompts (78%), and in 86% when its effect was clearly above noise.
- **The noise floor stopped over-claiming.** 481 of 741 phrases stayed below their noise floor, so the tool did not claim they changed the score.
- **Phrase types differed in these tests.** Removing a format instruction moved the score by 0.24 on average and the task itself by 0.23. Politeness moved it by 0.06 and tone by 0.04.
- **It surfaced behavior we didn't predict.** "dont guess" increased speculation (5 of 5 runs). "for github issues" kept a customer's name out of a bug report. "make up a salary" was refused.

07 What didn't work

- **Nothing to rank when a document did the work.** On the landlord and meeting tasks, removing any 2 phrases moved the score by under 0.07.
- **The AI judge couldn't count words.** Checked against code it agreed 88% of the time, but said yes to 93 answers that broke a word limit, and missed 6 leaked names or emails.
- **More reruns barely helped.** Nearly tripling the calls raised verdict agreement from 83% to 91%.
- **Our hallucination test failed.** The model never invented a salary, even with a $2,000 bonus in the posting as bait.

08 Limitations

- One model (Claude Haiku 5.5), on 8 tasks whose goals code can check.
- Test prompts were written by an AI imitating users, plus 32 by me. No study with real users yet.
- Each phrase is compared only with deleting it, not with a better wording.
- Phrases that do nothing alone are retested once as a group, not in every combination.
- Each noise floor comes from only 3 runs with the phrase and 2 without.

09 Where this could go

Training staff. Companies add their own tasks and documents, so employees practice on real work and see which habits change results.

Explainability outside AI teams. Anyone can see how the model read their own words.

Stretch goal: measure phrases at scale, then train a small local model or cache results, so an app can flag phrases unlikely to matter before a query is sent, with no AI call per check.

REFERENCES Zeiler & Fergus (2014), occlusion · Ribeiro et al. (2016), LIME · Lundberg & Lee (2017), SHAP · Petsiuk et al. (2018), deletion test · DeYoung et al. (2020), faithfulness · Alvarez-Melis & Jaakkola (2018), stability · Turpin et al. (2023), unfaithful model explanations